

Lambda Calculus

Week 4

Slides and materials based on the courses by
Noortje Venhuizen and Mareike Hartmann

Principle of Compositionality

“The meaning of a complex expression is a function of the meanings of its parts and of the syntactic rules by which they are combined”

(Partee, 1993)

$$\llbracket A \rrbracket = \mathbf{fa}(\llbracket B \rrbracket, \llbracket C \rrbracket)$$

- Compositional semantic construction:

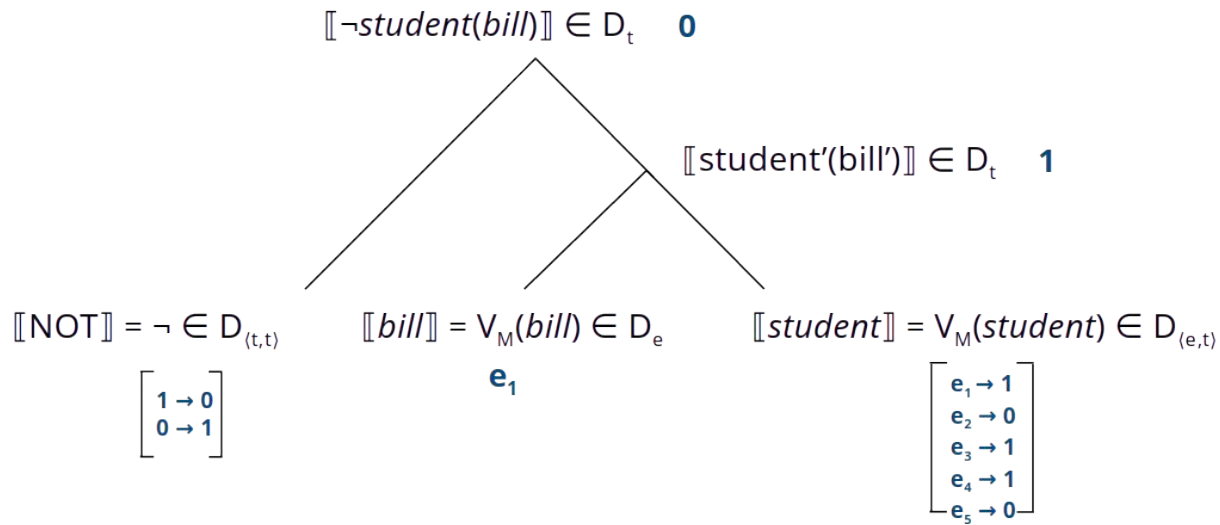
1. Define meaning representations for sub-expressions
2. Combine them in a principled manner to obtain a meaning representation for a complex expression.

$$\llbracket B \rrbracket \quad \llbracket C \rrbracket = \mathbf{fa}(\llbracket D \rrbracket, \llbracket E \rrbracket)$$

$$\llbracket D \rrbracket \quad \llbracket E \rrbracket$$

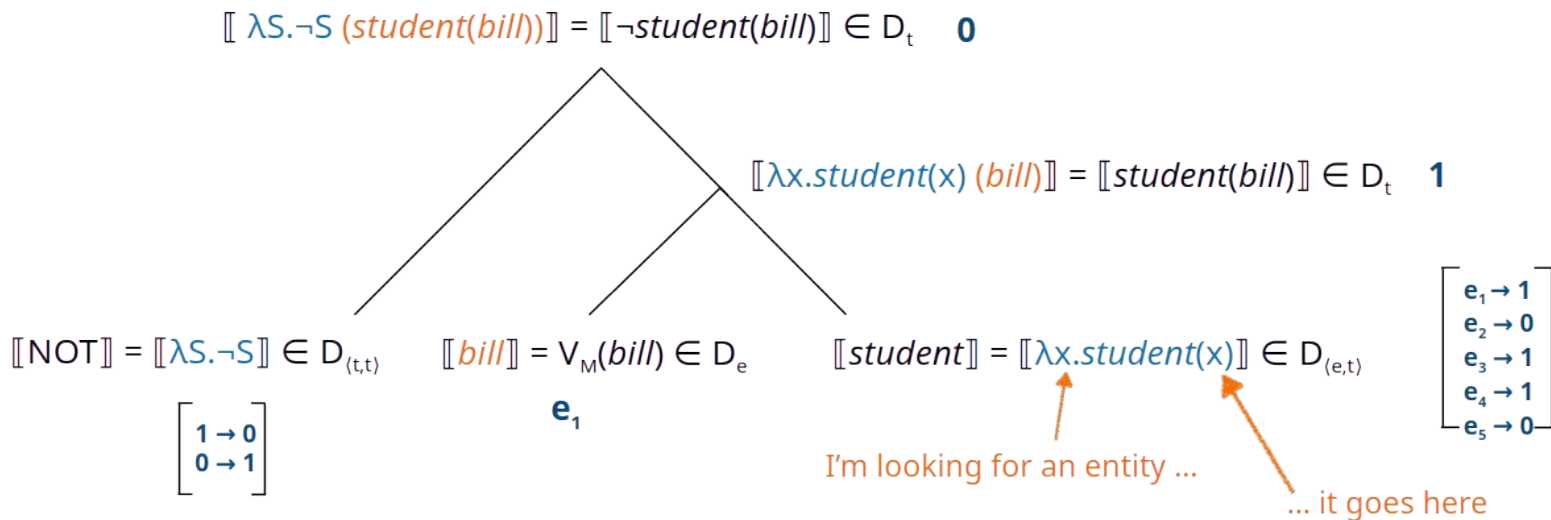
Compositionality: first try

“Bill is not a student” $\mapsto$ $[\text{NOT } [[\text{bill}]_{\text{NP}} [\text{student}]_{\text{VP}}]_{\text{S}}]_{\text{S}}$



Functions and arguments

“Bill is not a student” $\mapsto$ [NOT $[[\text{bill}]_{\text{NP}} [\text{student}]_{\text{VP}}]_{\text{S}}$]_S



Lambda expressions

- **Lambda expressions** are functions that consist of a set of lambda variables and a body
 - The **body** of a lambda expression is an *open** WFF:

$$[\text{Mary}_e [\text{sings and dances}]_{\langle e, t \rangle}] \mapsto \llbracket \lambda x (\text{sing}(x) \wedge \text{dance}(x))(\text{mary}) \rrbracket \in D_t$$

- Lambda expressions can themselves serve as arguments for functions (including other lambda expressions)

$$\llbracket [\text{Not smoking}]_{\langle e, t \rangle} [\text{is healthy}]_{\langle \langle e, t \rangle, t \rangle} \rrbracket \mapsto \llbracket \text{healthy}(\lambda y. \neg(\text{smoke}(y))) \rrbracket \in D_t$$

λ -abstraction

- Formal definition: if $\alpha \in WE_\sigma$ and $x \in VAR_\pi$, then $\lambda x(\alpha)$ is in $WE_{\langle \pi, \sigma \rangle}$
- **λ -abstraction**: the operation that transforms expressions of any type σ into a function $\pi \rightarrow \sigma$ (i.e. of the type $\langle \pi, \sigma \rangle$), where π is the type of the *λ -variable*
 - The scope of the λ -operator is the smallest WE to its right—wider scope must be indicated by brackets
 - We often use the “dot notation” $\lambda x.\varphi$ indicating that the λ -operator takes wide scope over φ

Interpretation of λ -expressions

- If $\alpha \in WE_\sigma$ and $v \in VAR_\pi$, then $\llbracket \lambda v \alpha \rrbracket^{M,g}$ is the function $f: D_\pi \rightarrow D_\sigma$ such that for all $d \in D_\pi$, $f(d) = \llbracket \alpha \rrbracket^{M,g[v/d]}$
- If the λ -expression is applied to an argument, we can simplify the interpretation:
 - $\llbracket \lambda v \alpha \rrbracket^{M,g}(\llbracket x \rrbracket^{M,g}) = \llbracket \alpha \rrbracket^{M,g[v/\llbracket x \rrbracket^{M,g}]}$
- Example: “*Bill is a student*”

$$\llbracket \lambda x (S(x))(b') \rrbracket^{M,g} = 1 \quad \text{iff} \quad \llbracket S(x) \rrbracket^{M,g'} = 1 \quad (\text{where } g' = g[x/\llbracket b' \rrbracket^{M,g}])$$

$$\llbracket \lambda x (S(x))(b') \rrbracket^{M,g} = \llbracket S(b') \rrbracket^{M,g}$$

Interpretation of λ -expressions

- For $\varphi \in WE_t$, $x \in VAR_\sigma$:

$$V_M(\lambda x. \varphi) = \{d \in D_\sigma \mid \llbracket \varphi \rrbracket^{M, g[x/d]}\}$$

- For example:

- $V_M(\lambda x. student(x) \wedge happy^*(x)) = V_M(student) \cap V_M(happy^*)$
- $V_M(\lambda x. blue^*(x) \vee green^*(x)) = V_M(blue^*) \cup V_M(green^*)$
- $V_M(\lambda x. see(john)(x)) = \{d \in D_e \mid (d, john) \in V_M(see)\}$
- $V_M(\lambda x. \forall y. eat(y)(x)) = \{d \in D_e \mid \llbracket \forall y. eat(y)(x) \rrbracket^{M, g[x/d]}\}$
- $V_M(\lambda x_e. like(john)(mary)) = ???$

β -reduction: function application in λ -calculus

- $\llbracket \lambda v(\alpha)(\beta) \rrbracket^{M,g} = \llbracket \alpha \rrbracket^{M,g[v/\llbracket \beta \rrbracket^{M,g}]}$
 - all (free) occurrences of the λ -variable (v) in α get the interpretation of β as their value
- This operation is called **β -reduction**
 - $\lambda v(\alpha)(\beta) \Leftrightarrow \alpha[v/\beta]$
 - where: $\alpha[v/\beta]$ is the result of replacing all free occurrences of v in α with β
 - **Warning: this equivalence is not unconditionally valid ...**

Variable capturing

- Are $\lambda v(\alpha)(\beta)$ and $\alpha[\beta/v]$ always equivalent?
 - $\lambda x(\text{sing}(x) \wedge \text{dance}(x))(\text{john}) \Leftrightarrow \text{sing}(\text{john}) \wedge \text{dance}(\text{john})$
 - $\lambda x(\text{sing}(x) \wedge \text{dance}(x))(y) \Leftrightarrow \text{sing}(y) \wedge \text{dance}(y)$ (where $y \in \text{VAR}_e$)
 - $\lambda x(\forall y.\text{know}(x)(y))(\text{john}) \Leftrightarrow \forall y.\text{know}(\text{john})(y)$
 - $\lambda x(\forall y.\text{know}(x)(y))(y) \not\Leftrightarrow \forall y.\text{know}(y)(y)$

Problem: y is not “free for x ” in $\forall y.\text{know}(x)(y)$

- Let x, y be variables of the same type, and let α be a WE of any type
 - **y is free for x** in α iff no free occurrence of x in α is in the scope of a quantifier or a λ -operator that binds y

Equivalence transformations in λ -calculus

- **β -conversion:** $\lambda v(\alpha)(\beta) \Leftrightarrow \alpha[v/\beta]$ (α with all instances of v replaced by β)
 - assuming all free variables in β are free for v in α
- **α -conversion:** $\lambda v.\alpha \Leftrightarrow \lambda w.\alpha[v/w]$ (α with all instances of v replaced by w)
 - assuming w is free for v in α
- **η -conversion:** $\lambda v.\alpha(v) \Leftrightarrow \alpha$

Quantifiers as λ -expressions

- “a student works” $\mapsto \exists x(\text{student}(x) \wedge \text{work}(x))$ $\vdash t$
 - “a student” $\mapsto \lambda P \exists x(\text{student}'(x) \wedge P(x))$ $\vdash \langle\langle e, t \rangle, t\rangle$
 - “a”, “some” $\mapsto \lambda Q \lambda P \exists x(Q(x) \wedge P(x))$ $\vdash \langle\langle e, t \rangle, \langle\langle e, t \rangle, t\rangle\rangle$
- “every student” $\mapsto \lambda P \forall x(\text{student}'(x) \rightarrow P(x))$ $\vdash \langle\langle e, t \rangle, t\rangle$
 - “every” $\mapsto \lambda Q \lambda P \forall x(Q(x) \rightarrow P(x))$ $\vdash \langle\langle e, t \rangle, \langle\langle e, t \rangle, t\rangle\rangle$
- “no student” $\mapsto \lambda P \neg \exists x(\text{student}(x) \wedge P(x))$ $\vdash \langle\langle e, t \rangle, t\rangle$
 - “no” $\mapsto \lambda Q \lambda P \neg \exists x(Q(x) \wedge P(x))$ $\vdash \langle\langle e, t \rangle, \langle\langle e, t \rangle, t\rangle\rangle$
- “someone” $\mapsto \lambda F \exists x(\text{person}(x) \wedge F(x))$ $\vdash \langle\langle e, t \rangle, t\rangle$
- “something” $\mapsto \lambda F \exists x.F(x)$ $\vdash \langle\langle e, t \rangle, t\rangle$

Interpretation of expressions of type $\langle\langle e, t \rangle, t\rangle$

- $something \in CON_{\langle\langle e, t \rangle, t\rangle}$, so $V_M(something) \in D_{\langle\langle e, t \rangle, t\rangle}$
- $D_{\langle\langle e, t \rangle, t\rangle}$ is the set of functions from $D_{\langle e, t \rangle}$ to D_t
 - i.e. the set of functions from $p(U_M)$ (the **powerset** of U_M) to $\{0, 1\}$, which in turn is equivalent to $p(p(U_M))$
- From $V_M(something) \in p(p(U_M))$ it follows that $V_M(something) \subseteq p(U_M)$
 - More specifically: $V_M(something) = \{S \subseteq U_M \mid S \neq \emptyset\}$, if U_M is a domain of individuals

Compositional construction

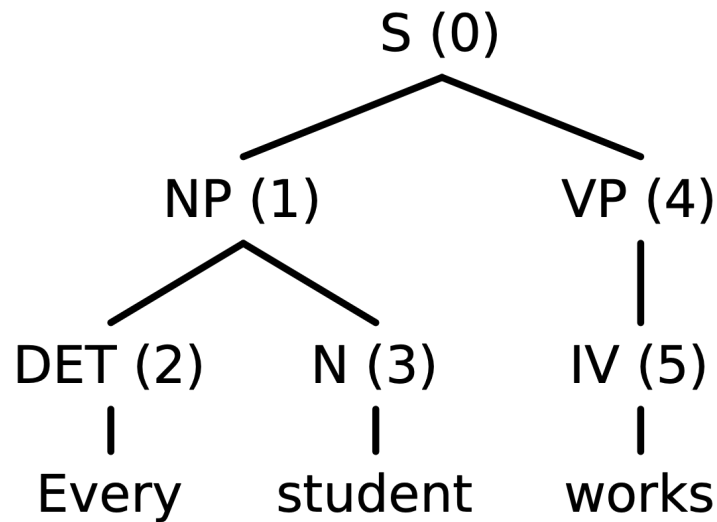
(2) $\lambda P \lambda Q \forall x (P(x) \rightarrow Q(x)) :: \langle \langle e, t \rangle, \langle \langle e, t \rangle, t \rangle \rangle$

(3) $\lambda y. student(y) \Leftrightarrow_{\eta} student :: \langle e, t \rangle$

(1) $\lambda P \lambda Q \forall x (P(x) \rightarrow Q(x))(student)$
 $\Leftrightarrow_{\beta} \lambda Q \forall x (student(x) \rightarrow Q(x)) :: \langle \langle e, t \rangle, t \rangle$

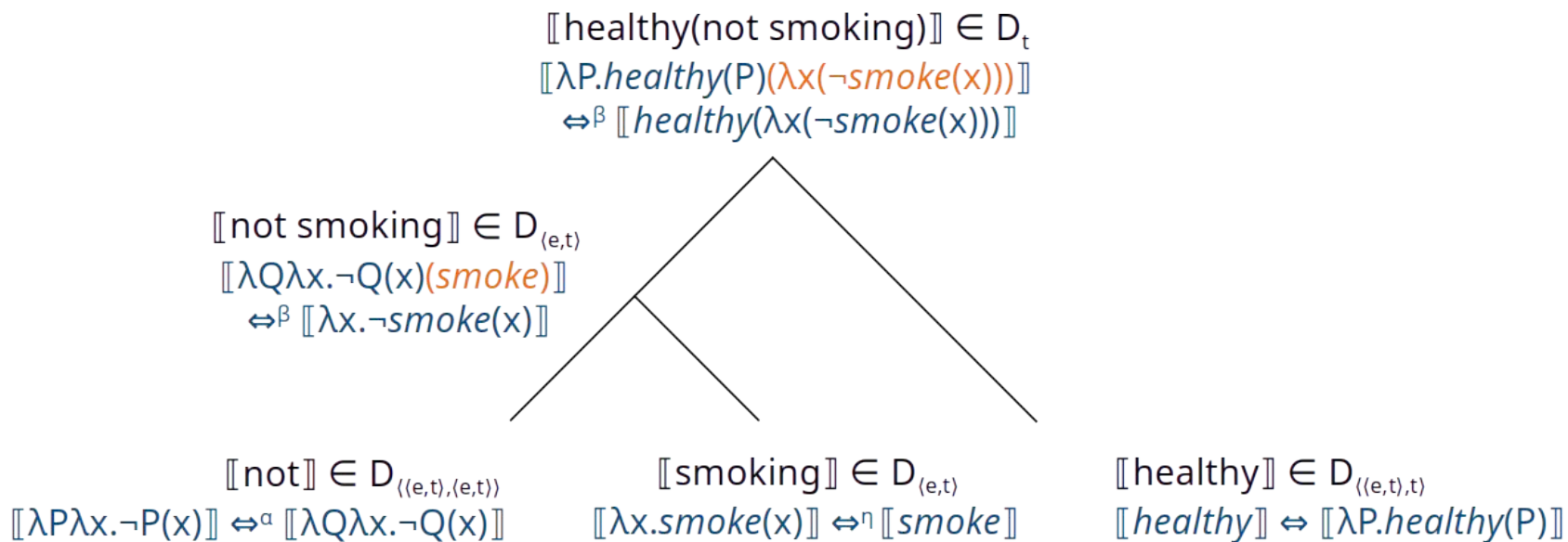
(4)/(5) $\lambda z. work(z) \Leftrightarrow_{\eta} work :: \langle e, t \rangle$

(0) $\lambda Q \forall x (student(x) \rightarrow Q(x))(work) \Leftrightarrow_{\beta} \forall x (student(x) \rightarrow work(x)) :: t$



Compositional construction

“*not smoking is healthy*” $\mapsto$ $\llbracket \text{not smoking} \rrbracket \llbracket \text{is healthy} \rrbracket$



Type Clash

- Problem: in natural language, quantified expressions occur with transitive verbs in both subject and object position
 - Example: “*someone reads a book*”

$$\begin{array}{ccc}
 & \text{read} :: \langle \mathbf{e}, \langle \mathbf{e}, \mathbf{t} \rangle \rangle & \text{a book} :: \langle \langle \mathbf{e}, \mathbf{t} \rangle, \mathbf{t} \rangle \\
 \hline
 \text{someone} :: \langle \langle \mathbf{e}, \mathbf{t} \rangle, \mathbf{t} \rangle & & ?? :: ?? \\
 \hline
 & ?? :: \mathbf{t} &
 \end{array}$$

- Solution: reverse functor-argument relation
 - Logical form: *someone(read(a book))*
 - Use **type raising** to adjust the type of the transitive verb: *read* _{$\langle \langle \langle \mathbf{e}, \mathbf{t} \rangle, \mathbf{t} \rangle, \langle \mathbf{e}, \mathbf{t} \rangle \rangle$}

Type Raising

- What if we just change the type of the transitive verb?

- “read” $\rightarrow read \in CON_{\langle \langle \langle e, t \rangle, t \rangle, \langle e, t \rangle \rangle}$

$\llbracket \text{someone reads a book} \rrbracket =$

$\llbracket \lambda F \exists x (\text{person}(x) \wedge F(x)) (\text{read}(\lambda P \exists y (\text{book}'(y) \wedge P(y)))) \rrbracket$

$\Leftrightarrow_{\beta} \llbracket \exists x (\text{person}(x) \wedge \text{read}(\lambda P \exists y (\text{book}'(y) \wedge P(y)))(x)) \rrbracket$

- Stuck! We need a more explicit λ -term:

- $read \leftarrow \lambda Q \lambda z. Q(\lambda x (\text{read}^*(x)(z))) \in WE_{\langle \langle \langle e, t \rangle, t \rangle, \langle e, t \rangle \rangle}$ (*type raising*)
- where $\text{read}^* \in WE_{\langle e, \langle e, t \rangle \rangle}$ is the “underlying” first-order relation

Type Raising (Cont.)

- Specifically, given $f: \pi \rightarrow \sigma$, $g: \sigma \rightarrow \delta$, we can *type raise* f to get $\uparrow(f): (\sigma \rightarrow \delta) \rightarrow (\pi \rightarrow \delta)$, so that f takes g as an argument (this is basically just function composition):
 - $\uparrow(f) = \lambda h_{(\sigma \rightarrow \delta)} \lambda x_{\pi}. h(f(x))$
 - $\uparrow(f)(g) = \lambda x_{\pi}. g(f(x)): \pi \rightarrow \delta$
- Given $c: \sigma$, $g: \sigma \rightarrow \delta$, we can consider c as a function $c: \emptyset \rightarrow \sigma$
 - $\uparrow(c): (\sigma \rightarrow \delta) \rightarrow (\emptyset \rightarrow \delta)$
 - $\uparrow(c): \lambda h_{(\sigma \rightarrow \delta)}. h(c)$
 - $\uparrow(c)(g) = g(c): \emptyset \rightarrow \delta \Leftrightarrow \delta$

Type Raising

someone reads a book $\mapsto$ someone($\text{reads}(\text{a book})$)

$$\lambda F \exists x (\text{person}(x) \wedge F(x)) (\lambda Q \lambda z (Q(\lambda w (\text{read}^*(w)(z)))) (\lambda P \exists y (\text{book}(y) \wedge P(y))))$$
$$\Leftrightarrow_{\beta} \lambda F \exists x (\text{person}(x) \wedge F(x)) (\lambda z (\lambda P \exists y (\text{book}(y) \wedge P(y)) (\lambda w (\text{read}^*(w)(z)))))$$
$$\Leftrightarrow_{\beta} \lambda F \exists x (\text{person}(x) \wedge F(x)) (\lambda z (\exists y (\text{book}(y) \wedge \lambda w (\text{read}^*(w)(z))(y))))$$
$$\Leftrightarrow_{\beta} \lambda F \exists x (\text{person}(x) \wedge F(x)) (\lambda z (\exists y (\text{book}(y) \wedge \text{read}^*(y)(z))))$$
$$\Leftrightarrow_{\beta} \exists x (\text{person}(x) \wedge \lambda z (\exists y (\text{book}(y) \wedge \text{read}^*(y)(z)))(x))$$
$$\Leftrightarrow_{\beta} \exists x (\text{person}(x) \wedge \exists y (\text{book}(y) \wedge \text{read}^*(y)(x)))$$